



파머완 12주차

Chapter 8. 텍스트 분석

NLP (National Language Processing): 머신이 인간의 언어를 이해하고 해석하는 데 더 중점

- 기계 번역
- 질의응답 시스템

→ 텍스트 분석을 향상하게 하는 기반 기술, 머신러닝 필요

텍스트 분석 (텍스트 마이닝): 비정형 텍스트에서 의미 있는 정보를 추출하는 것에 더 중점

- 머신러닝, 언어 이해, 통계 등 활용
- 비즈니스 인텔리전스, 예측 분석 등 분석 작업 수행

- **텍스트 분류(Text Classification):** Text Categorization이라고도 합니다. 문서가 특정 분류 또는 카테고리에 속하는 것을 예측하는 기법을 통칭합니다. 예를 들어 특정 신문 기사 내용이 연예/정치/사회/문화 중 어떤 카테고리에 속하는지 자동으로 분류하거나 스팸 메일 검출 같은 프로그램이 이에 속합니다. 지도학습을 적용합니다.
- **감성 분석(Sentiment Analysis):** 텍스트에서 나타나는 감정/판단/민감/의견/기분 등의 주관적인 요소를 분석하는 기법을 통칭합니다. 소셜 미디어 감정 분석, 영화나 제품에 대한 긍정 또는 리뷰, 여론조사 의견 분석 등의 다양한 영역에서 활용됩니다. Text Analytics에서 가장 활발하게 사용되고 있는 분야입니다. 지도학습 방법뿐만 아니라 비지도학습을 이용해 적용할 수 있습니다.
- **텍스트 요약(Summarization):** 텍스트 내에서 중요한 주제나 중심 사상을 추출하는 기법을 말합니다. 대표적으로 토픽 모델링(Topic Modeling)이 있습니다.
- **텍스트 군집화(Clustering)와 유사도 측정:** 비슷한 유형의 문서에 대해 군집화를 수행하는 기법을 말합니다. 텍스트 분류를 비지도학습으로 수행하는 방법의 일환으로 사용될 수 있습니다. 유사도 측정 역시 문서들간의 유사도를 측정해 비슷한 문서끼리 모을 수 있는 방법입니다.

1. 텍스트 분석 이해

텍스트 분석: 비정형 데이터인 텍스트 분석

머신러닝 알고리즘은 숫자형의 피쳐 기반 데이터만 입력받을 수 있기 때문에 비정형 텍스트 데이터를 어떻게 피쳐 형태로 추출하고 추출된 피쳐에 의미 있는 값을 부여하는가가 중요

피쳐 벡터화, 피쳐 추출

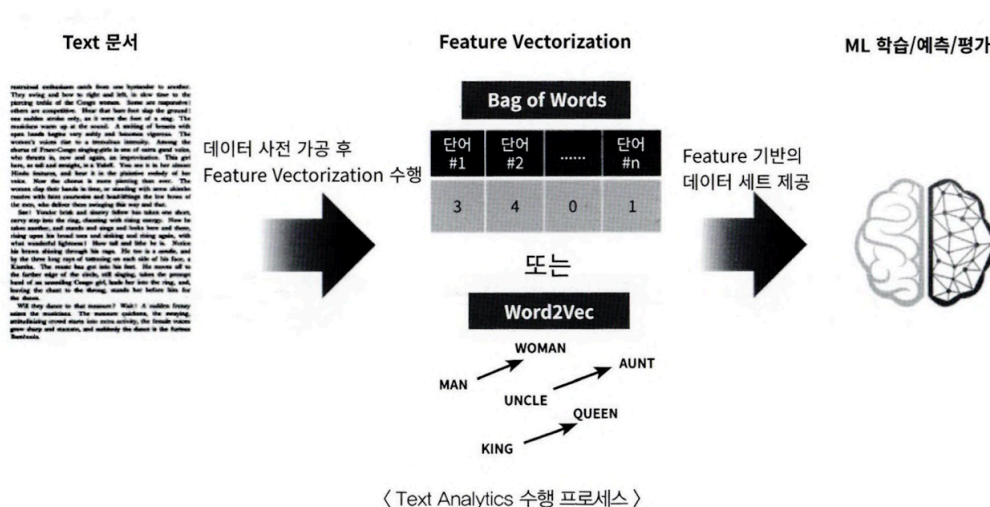
텍스트를 word 기반의 다수의 피쳐로 추출하고 이 피쳐에 단어 빈도수와 같은 숫자 값을 부여하면 텍스트는 단어의 조합인 벡터값으로 표현될 수 있음

ex) BOW (Bag of Words), Word2Vec

텍스트 분석 수행 프로세스

텍스트 전처리 → 피쳐 벡터화/추출 → ML 모델 수립 및 학습/예측/평가

1. 텍스트 사전 준비작업(텍스트 전처리): 텍스트를 피쳐로 만들기 전에 미리 클렌징, 대/소문자 변경, 특수문자 삭제 등의 클렌징 작업, 단어(Word) 등의 토큰화 작업, 의미 없는 단어(Stop word) 제거 작업, 어근 추출(Stemming/Lemmatization) 등의 텍스트 정규화 작업을 수행하는 것을 통칭합니다.
2. 피쳐 벡터화/추출: 사전 준비 작업으로 가공된 텍스트에서 피쳐를 추출하고 여기에 벡터 값을 할당합니다. 대표적인 방법은 BOW와 Word2Vec이 있으며, BOW는 대표적으로 Count 기반과 TF-IDF 기반 벡터화가 있습니다.
3. ML 모델 수립 및 학습/예측/평가: 피쳐 벡터화된 데이터 세트에 ML 모델을 적용해 학습/예측 및 평가를 수행합니다.



파이썬 기반의 NLP, 텍스트 분석 패키지

NLTK, gensim, spaCy

- NLTK(Natural Language Toolkit for Python): 파이썬의 가장 대표적인 NLP 패키지입니다. 방대한 데이터 세트와 서브 모듈을 가지고 있으며 NLP의 거의 모든 영역을 커버하고 있습니다. 많은 NLP 패키지가 NLTK의 영향을 받아 작성되고 있습니다. 수행 속도 측면에서 아쉬운 부분이 있어서 실제 대량의 데이터 기반에서는 제대로 활용되지 못하고 있습니다.
- Gensim: 토픽 모델링 분야에서 가장 두각을 나타내는 패키지입니다. 오래전부터 토픽 모델링을 쉽게 구현할 수 있는 기능을 제공했으며, Word2Vec 구현 등의 다양한 신기능도 제공합니다. SpaCy와 함께 가장 많이 사용되는 NLP 패키지입니다.
- SpaCy: 뛰어난 수행 성능으로 최근 가장 주목을 받는 NLP 패키지입니다. 많은 NLP 애플리케이션에서 SpaCy를 사용하는 사례가 늘고 있습니다.

- 사이킷런은 NLP 패키지에 특화된 라이브러리는 가지고 있지 않지만 텍스트를 일정 수준으로 가공하고 머신러닝 알고리즘에 텍스트 데이터를 피처로 처리하기 위한 편리한 기능 제공
- NLP 전용 패키지와 함께 결합해 사용

2. 텍스트 사전 준비 작업 (텍스트 전처리) — 텍스트 정규화

클렌징

클렌징: 텍스트에서 분석에 방해가 되는 불필요한 문자, 기호 등을 제거

텍스트 토큰화

문장 토큰화: 문장의 마침표(.), 개행문자(\n) 등 문장의 마지막을 뜻하는 기호에 따라 분리

단어 토큰화: 문장을 단어로 토큰화 → 공백, 콤마(,), 마침표(.) 등으로 단어 분리

- Bag of Word와 같이 단어의 순서가 중요하지 않은 경우 문장 토큰화를 사용하지 않고 단어 토큰화만 사용해도 됨
- 시맨틱적인 의미가 중요할 때 사용

스톱 워드 제거

스톱 워드: 분석에 큰 의미가 없는 단어

is, the, a, will 등의 스톱워드는 문맥적으로 큰 의미가 없지만 텍스트에 많이 나타나므로 오히려 중요한 단어로 인지될 수 있기 때문에 사전에 제거

언어별로 스톱 워드가 목록화 되어있음

Stemming & Lemmatization

Stemming & Lemmatization: 문법적, 의미적으로 변화하는 단어의 원형을 찾는 것

Stemming: 더 단순화된 방법을 적용해 원래 단어에서 일부 철자가 훼손된 어근 단어를 추출

Lemmatization: 문법적인 요소와 더 의미적인 부분을 감안해 정확한 철자로 된 어근 단어를 추출

NLTK가 제공하는 모델들: Porter, Lancaster, Snowball Stemmer, WordNetLemmatizer 등

3. Bag of Words - BOW

Bag of Words: 문서가 가지는 모든 단어를 문맥이나 순서를 무시하고 일괄적으로 단어에 대해 빈도 값을 부여해 피쳐 값을 추출하는 모델



〈 Bag of Words 모델 〉

2개의 문장에서 Bag of words의 단어 수 기반으로 피쳐 추출 예시

- 문장 1: 'My wife likes to watch baseball games and my daughter likes to watch baseball games too'
- 문장 2: 'My wife likes to play baseball'

- 문장 1과 문장 2에 있는 모든 단어에서 중복을 제거하고 각 단어(feature 또는 term)를 칼럼 형태로 나열합니다. 그리고 나서 각 단어에 고유의 인덱스를 다음과 같이 부여합니다.

'and':0, 'baseball':1, 'daughter':2, 'games':3, 'likes':4, 'my':5, 'play':6, 'to':7, 'too':8, 'watch':9, 'wife':10

- 개별 문장에서 해당 단어가 나타나는 횟수(Occurrence)를 각 단어(단어 인덱스)에 기재합니다. 예를 들어 baseball은 문장 1, 2에서 총 2번 나타나며, daughter는 문장 1에서만 1번 나타납니다.

	Index 0	Index 1	Index 2	Index 3	Index 4	Index 5	Index 6	Index 7	Index 8	Index 9	Index 10
	and	baseball	daughter	games	likes	my	play	to	too	watch	wife
문장 1	1	2	1	2	2	2		2	1	2	1
문장 2		1			1	1	1	1			1

→ 문장 1에서 baseball은 2회 나타남

장점: 쉽고 빠른 구축, 여러 분야에서 활용도 높음

단점: 문맥 의미 반영 부족, 희소 행렬 문제

BOW 피쳐 벡터화

피쳐 벡터화: 각 문서의 텍스트를 단어로 추출해 피쳐로 할당하고, 각 단어의 발생 빈도와 같은 값을 이 피쳐에 값으로 부여해 각 문서를 이 단어 피쳐의 발생 빈도 값으로 구성된 벡터로 만드는 기법

BOW 모델에서 피쳐 벡터화: 모든 문서에서 모든 단어를 칼럼 형태로 나열하고 각 문서에서 해당 단어의 횟수나 정규화된 빈도를 값으로 부여하는 데이터 세트 모델로 변경하는 것

M개의 텍스트 문서 + N개의 단어 → M X N개의 단어 피쳐로 이루어진 행렬

카운트 기반의 벡터화

각 문서에서 해당 단어가 나타나는 횟수, 즉 Count를 부여하는 경우

언어의 특성상 문장에서 자주 사용될 수밖에 없는 단어까지 높은 값을 부여하게 됨

TF-IDF (Term Frequency - Inverse Document Frequency) 기반의 벡터화

개별 문서에서 자주 나타나는 단어에 높은 가중치를 주되, 모든 문서에서 전반적으로 자주 나타나는 단어에 대해서는 페널티를 줌

어떤 문서에서 특정 단어가 자주 나타나면 그 단어는 해당 문서를 특징짓는 중요 단어일 수 있지만 그 단어가 다른 문서에도 자주 나타나는 단어라면 그 단어는 언어 특성상 자주 사용되는 단어일 가능성이 높음



한 개의 문서(Document)



모든 문서들(Corpus)

The	Matrix	is	nothing	but	an	advertising	gimmick
40	5	50	12	20	45	3	2

The	Matrix	is	nothing	but	an	advertising	gimmick
2000	190	2300	500	1200	3000	52	12

$$TFIDF_i = TF_i * \log \frac{N}{DF_i}$$

TF_i = 개별 문서에서의 단어 i 빈도
 DF_i = 단어 i를 가지고 있는 문서 개수
 N = 전체 문서 개수

사이킷런의 Count 및 TF-IDF 벡터화 구현: CountVectorizer, TfidfVectorizer

CountVectorizer 클래스

카운트 기반의 벡터화를 구현한 클래스

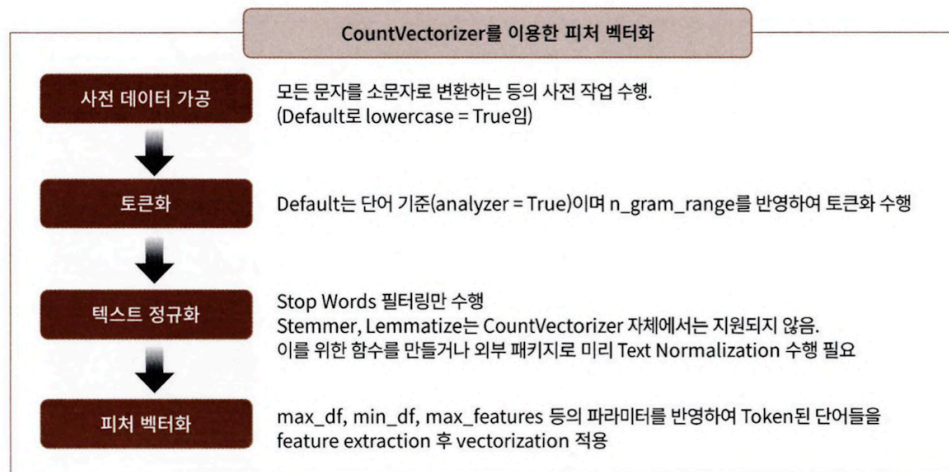
피처 벡터화 이외에도 소문자 일괄 변환, 토큰화, 스톱 워드 필터링 등의 텍스트 전처리도 함께 수행

입력 파라미터

파라미터 명	파라미터 설명
max_df	전체 문서에 걸쳐서 너무 높은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터입니다. 너무 높은 빈도수를 가지는 단어는 스톱 워드와 비슷한 문법적인 특성으로 반복적인 단어일 가능성이 높기에 이를 제거하기 위해 사용됩니다. max_df = 100과 같이 정수 값을 가지면 전체 문서에 걸쳐 100개 이하로 나타나는 단어만 피처로 추출합니다. Max_df = 0.95와 같이 부동소수점 값(0.0 ~ 1.0)을 가지면 전체 문서에 걸쳐 빈도수 0~95%까지의 단어만 피처로 추출하고 나머지 상위 5%는 피처로 추출하지 않습니다.
min_df	전체 문서에 걸쳐서 너무 낮은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터입니다. 수백~수천 개의 전체 문서에서 특정 단어가 min_df에 설정된 값보다 적은 빈도수를 가진다면 이 단어는 크게 중요하지 않거나 가비지(garbage)성 단어일 확률이 높습니다. min_df = 2와 같이 정수 값을 가지면 전체 문서에 걸쳐서 2번 이하로 나타나는 단어는 피처로 추출하지 않습니다. min_df = 0.02와 같이 부동소수점 값(0.0 ~ 1.0)을 가지면 전체 문서에 걸쳐서 하위 2% 이하의 빈도수를 가지는 단어는 피처로 추출하지 않습니다.
max_features	추출하는 피처의 개수를 제한하며 정수로 값을 지정합니다. 가령 max_features = 2000으로 지정할 경우 가장 높은 빈도를 가지는 단어 순으로 정렬해 2000개까지만 피처로 추출합니다.
stop_words	'english'로 지정하면 영어의 스톱 워드로 지정된 단어는 추출에서 제외합니다.
n_gram_range	Bag of Words 모델의 단어 순서를 어느 정도 보강하기 위한 n_gram 범위를 설정합니다. 튜플 형태로 (범위 최솟값, 범위 최댓값)을 지정합니다. 예를 들어 (1, 1)로 지정하면 토큰화된 단어를 1개씩 피처로 추출합니다. (1, 2)로 지정하면 토큰화된 단어를 1개씩(minimum 1), 그리고 순서대로 2개씩(maximum 2) 묶어서 피처로 추출합니다.
analyzer	피처 추출을 수행한 단위를 지정합니다. 당연히 디폴트는 'word'입니다. Word가 아니라 character의 특정 범위를 피처로 만드는 특정한 경우 등을 적용할 때 사용됩니다.
token_pattern	토큰화를 수행하는 정규 표현식 패턴을 지정합니다. 디폴트 값은 '\b\w+\b'로, 공백 또는 개행 문자 등으로 구분된 단어 분리자(\b) 사이의 2문자(문자 또는 숫자, 즉 영숫자) 이상의 단어(word)를 토큰으로 분리합니다. analyzer= 'word'로 설정했을 때만 변경 가능하나 디폴트 값을 변경할 경우는 거의 발생하지 않습니다.
tokenizer	토큰화를 별도의 커스텀 함수로 이용시 적용합니다. 일반적으로 CountTokenizer 클래스에서 어근 변환 시 이를 수행하는 별도의 함수를 tokenizer 파라미터에 적용하면 됩니다.

CountVectorizer 클래스를 이용한 피처 벡터화 방법

1. 영어의 경우 모든 문자를 소문자로 변경
2. 디폴트로 단어 기준으로 n_gram_range를 반영해 각 단어 토큰화
3. 텍스트 정규화 수행
4. max_df, min_df, max_features 등의 파라미터를 이용해 토큰화된 단어를 피처로 추출하고 단어 빈도수 벡터 값을 적용



BOW 벡터화를 위한 희소 행렬

희소 행렬: 대규모 행렬의 대부분의 값을 0이 차지하는 행렬

모든 문서에 있는 단어를 추출해 이를 피쳐로 벡터화하는 방법은 많은 피쳐 칼럼을 만들 수 밖에 없고, 이 행렬의 값은 대부분 0

→ BOW 형태를 가진 언어 모델의 피쳐 벡터화는 대부분 희소 행렬

← 수십만 개의 칼럼 →

	단어 1	단어 2	단어 3		단어 1000		단어 2000		단어 10000		단어 20000		단어 100000
문서1	1	2	2	0	0	0	0	0	0	0	0	0	0
문서2	0	0	1	0	0	1	0	0	1	0	0	0	1
문서 ...													
문서 10000	0	1	3	0	0	0	0	0	0	0	0	0	0

수천~수만 개 레코드

BOW의 Vectorization 모델은 너무 많은 0값이 메모리 공간에 할당되어 많은 메모리 공간이 필요하며 연산 시에도 데이터 액세스를 위한 많은 시간이 소모됩니다.

메모리 공간을 많이 차지하고, 연산 시에도 데이터 액세스를 위한 시간이 많이 소모됨

→ 물리적으로 적은 메모리 공간을 차지할 수 있도록 변환해야함

→ COO 형식 & CSR 형식

희소 행렬 — COO 형식

COO (Coordinate: 좌표) 형식

0이 아닌 데이터만 별도의 데이터 배열에 저장하고, 그 데이터가 가리키는 행과 열의 위치를 별도의 배열로 저장하는 방식

Scipy 사용: sparse 패키지가 희소 행렬 변환을 위한 다양한 모듈 제공

희소 행렬 — CSR 형식

CSR (Compressed Sparse Row) 형식

COO 형식이 행과 열의 위치를 나타내기 위해서 반복적인 위치 데이터를 사용해야 하는 문제점을 해결한 방식

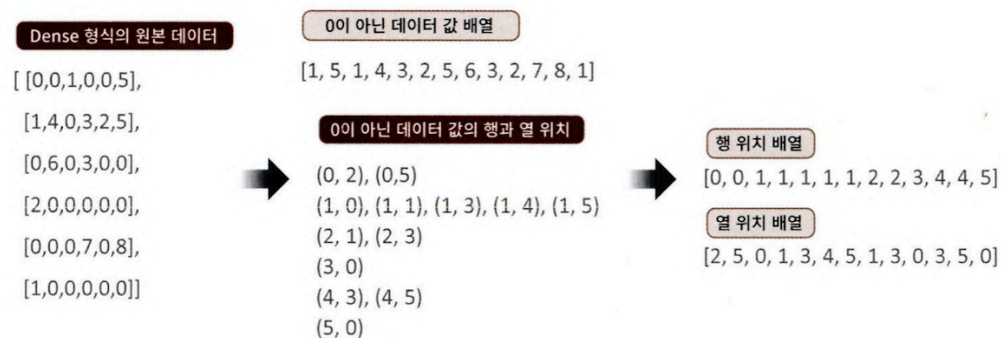
```
[ [0, 0, 1, 0, 0, 5], [1, 4, 0, 3, 2, 5], [0, 6, 0, 3, 0, 0], [2, 0, 0, 0, 0, 0], [0, 0, 0, 7, 0, 8], [1, 0, 0, 0, 0, 0] ]
```

이 2차원 배열을 COO 형식으로 변환하면,

데이터 배열: [1, 5, 1, 4, 3, 2, 5, 6, 3, 2, 7, 8, 1]

행 위치 배열: [0, 0, 1, 1, 1, 1, 1, 2, 2, 3, 4, 4, 5]

열 위치 배열: [2, 5, 0, 1, 3, 4, 5, 1, 3, 0, 3, 5, 0]



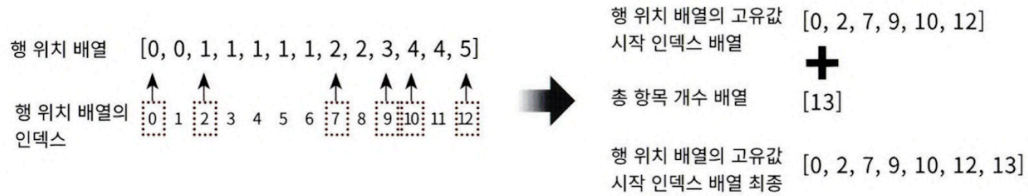
행 위치 배열을 보면 순차적인 같은 값이 반복적으로 나타나는데, 행 위치 배열의 고유한 값의 시작 위치만 표기하는 방법으로 이러한 반복 제거 가능 → 위치의 위치 표기

CSR (Compressed Sparse Row): 행 위치 배열 내에 있는 고유한 값의 시작 위치만 다시 별도의 위치 배열로 가지는 변환 방식

행 위치 배열 [0, 0, 1, 1, 1, 1, 1, 2, 2, 3, 4, 4, 5] → [0, 2, 7, 9, 10, 12]

데이터의 총 항목 개수를 배열에 추가하면 최종적으로 [0, 2, 7, 9, 10, 12, 13]

→ 고유 값의 시작 위치만 알고 있으면 얼마든지 행 위치 배열을 다시 만들 수 있기에 COO 방식보다 메모리가 적게 들고 빠른 연산 가능



car_matrix 클래스 이용

5. 감성 분석

감성 분석 소개

감성 분석 (Sentiment Analysis)

- 문서의 주관적인 감성/의견/감정/기분 등을 파악하기 위한 방법으로 소셜 미디어, 여론 조사, 온라인 리뷰, 피드백 등 다양한 분야에서 활용
- 문서 내 텍스트가 나타내는 여러 가지 주관적인 단어와 문맥을 기반으로 감성 수치를 계산
- 긍정 감성 지수 & 부정 감성 지수
- 지도학습 & 비지도학습 방식으로 나눌 수 있음

- 지도학습은 학습 데이터와 타겟 레이블 값을 기반으로 감성 분석 학습을 수행한 뒤 이를 기반으로 다른 데이터의 감성 분석을 예측하는 방법으로 일반적인 텍스트 기반의 분류와 거의 동일합니다.
- 비지도학습은 'Lexicon'이라는 일종의 감성 어휘 사전을 이용합니다. Lexicon은 감성 분석을 위한 용어와 문맥에 대한 다양한 정보를 가지고 있으며, 이를 이용해 문서의 긍정적, 부정적 감성 여부를 판단합니다.

지도학습 기반 감성 분석 → 텍스트 기반의 이진 분류

비지도학습 기반 감성 분석 소개

Lexicon 기반

많은 감성 분석용 데이터는 결정된 레이블 값을 가지고 있지 않음

감성 지수: 긍정 감성 또는 부정 감성의 정도를 의미하는 수치, 단어의 위치나 주변 단어, 문맥, POS (Part of Speech)등을 참고해 결정

NLTK 패키지의 **WordNet:** 방대한 영어 어휘 사전

NLP 패키지: 시맨틱(문맥상 의미)을 프로그램적으로 인터페이스할 수 있는 방법 제공

Synset: 단순한 하나의 단어가 아니라 그 단어가 가지는 문맥, 시맨틱 정보를 제공하는 WordNet의 핵심 개념

NLTK를 포함한 대표적인 감성 사전

- SentiWordNet: NLTK 패키지의 WordNet과 유사하게 감성 단어 전용의 WordNet을 구현한 것입니다. WordNet의 Synset 개념을 감성 분석에 적용한 것입니다. WordNet의 Synset별로 3가지 감성 점수(sentiment score)를 할당합니다. 긍정 감성 지수, 부정 감성 지수, 객관성 지수가 그것입니다. 긍정 감성 지수는 해당 단어가 감성적으로 얼마나 긍정적인가를, 부정 지수는 얼마나 감성적으로 부정적인가를 수치로 나타낸 것입니다. 객관성 지수는 긍정/부정 감성 지수와 완전히 반대되는 개념으로 단어가 감성과 관계없이 얼마나 객관적인지를 수치로 나타낸 것입니다. 문장별로 단어들의 긍정 감성 지수와 부정 감성 지수를 합산하여 최종 감성 지수를 계산하고 이에 기반해 감성이 긍정인지 부정인지를 결정합니다.
- VADER: 주로 소셜 미디어의 텍스트에 대한 감성 분석을 제공하기 위한 패키지입니다. 뛰어난 감성 분석 결과를 제공하며, 비교적 빠른 수행 시간을 보장해 대용량 텍스트 데이터에 잘 사용되는 패키지입니다.
- Pattern: 예측 성능 측면에서 가장 주목받는 패키지입니다. 아쉽게도 현재 기준으로 파이썬 3.X 버전에서 호환이 되지 않고, 파이썬 2.X 버전에서만 동작합니다. 이 책에서는 사용 예제를 소개하지는 않습니다만, 감성 분석에 관심이 많은 사람이려면 적용해 보는 것도 좋습니다.

SentiWordNet을 이용한 감성 분석

1. 문서를 문장 단위로 분해
2. 다시 문장을 단어 단위로 토큰화하고 품사 태깅
3. 품사 태깅된 단어 기반으로 synset 객체와 senti_synset 객체를 생성
4. senti_synset 객체에서 긍정 감성/부정 감성 지수를 구하고 이를 모두 합산해 특정 임계치 값 이상일 때 긍정 감성으로, 그렇지 않을 때는 부정 감성으로 결정

VADER를 이용한 감성 분석

소셜 미디어의 감성 분석 용도로 만들어진 룰 기반의 Lexicon

SentimentIntensityAnalyzer 클래스 이용

문서별로 polarity_scores() 메서드를 호출해 감성 점수를 구한 뒤, 해당 문서의 감성 점수가 특정 임계값 이상이면 긍정, 그렇지 않으면 부정

polarity_scores() 메서드: 딕셔너리 형태의 감성 점수 반환

'neg': 부정 감성 지수

'neu': 중립적인 감성 지수

'pos' 긍정 감성 지수

compound: neg, neu, pos score를 적절히 조합해 -1에서 1 사이의 감성 지수를 표현한 값